Assignment due date as noted on Quercus
By electronic submission only

This assignment is individual

Student Name (Last, First):

Student number:

Student UtorID:

I hereby affirm that all the solutions we provide, both in writing and in code, for this assignment are my own. I have properly cited and noted any reference material we used to arrive at this solution, and have not shared our work with anyone else. I have followed the course's policy regarding use of A.I. coding tools, and have attached all relevant documentation of my use of A.I. (if any) in completing the programming components of the assignment.

_____
(signature)

*Check **one** of the options below:*

☐ *I have **not** used A.I. tools*

☐ *I **have used** A.I. tools and included the relevant logs*

(note: 3 marks penalty if any of the above information is missing)

At this point we are ready to tackle the problem of rendering a scene in 3D. This requires understanding of the complete process of image formation, object representation and manipulation using affine transformations, coordinate frames and conversions, and projection.

Assignments 2 and 3 will be dedicated to helping you strengthen and refine your understanding of these topics by building a fully operational ray tracer. Within a few weeks your code will be rendering high quality scenes with realistic illumination!

Learning Objectives – after completing this assignment you should be able to:

- Set up a scene using canonical objects and affine transformations, illuminated by point light sources.

- Set up a camera, and determine the correct transformation that determines which points in the scene map to which pixels in the image.

- Determine the parameters of a ray going from the camera's center of projection, through any pixel in the image, and into the scene.

- Set up the equations that give the intersection between rays from the camera and objects in the scene. Along with the process to determine local geometry including surface normal.

- Use local geometry at an intersection between an object and a ray to evaluate the components of the Phong illumination model. Explain what each of the components contributes to the appearance of an object.

- Understand, explain, and simulate the basic geometry of light that yields shadows and global specular reflection via recursive ray tracing.

Skills Developed:

- Implementing the full image formation pipeline – from objects in a scene to pixel colours

- Using affine transforms to manipulate objects and create scenes, and to easily handle ray-object intersections for deformed objects.

- Handling illumination effects such as shadows and global specular reflections.

- Developing and debugging software that works with light rays, objects, transforms, and images.

Reference material:

Lecture notes for camera projections, transformations, and simple ray tracing.

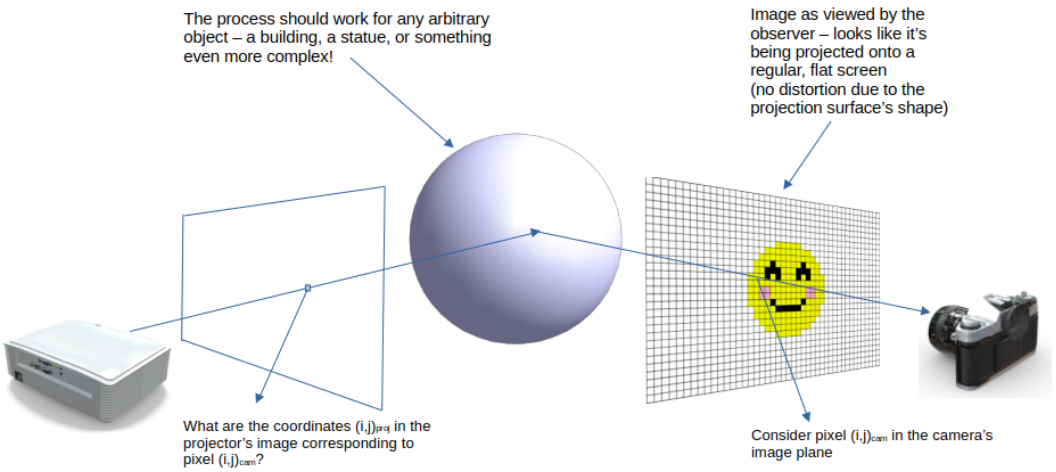
The detailed comments in the starter code. The starter code ***already provides a lot of the simple building blocks you will need***. Don't waste time implementing functionality already there.

Your TA and instructor! Don't get stuck! Seek help early!

Part 1 – Projection mapping

Projection mapping is a technique used to generate and project an image onto a surface that is ***not flat*** – you may have seen this used to project images and video onto buildings, monuments, or other such surfaces. The key to projection mapping is to sigure out figure out what image the projector must show so that when projected onto the surface, it looks to the observer as if the image was being projected onto a flat surface (with no distortion due to surface shape).

A simplified setup is shown below, where the projection surface is a simple sphere (in general, the surface may be something very complex, but we will study the problem first with a simple shape).



Work this out by yourself, without any A.I. assistance – come to office hours with your TA or instructor if you would like help! We can help you without giving away the answer so you can fully understand the geometry involved.

1) [5 marks] Assuming that the observer camera is at the world's origin, the projection surface is a sphere with center at $\bar{c} = [c_x \;\; c_y \;\; c_z]$ and has radius ***r***, the image plane has window size of 2 along both the ***x*** and ***y*** directions, and the image has a resolution of 500 pixels in each dimension.

   Write down the equations required to determine the ***location on the projection surface*** that produces the colour viwed by the camera at pixel $(i,j)_{cam}$.

2) [5 marks] Given the point $\bar{p}_s = [p_x \;\; p_y \;\; p_z]$ corresponding to the surface point you found in 1), and considering the projector's center of projection is at $\bar{e}_p = [e_x \;\; e_y \;\; e_z]$, has a focal length of ***f***, is *gazing* at the center of the sphere, and has an up vector $\bar{t} = [0 \;\; 1 \;\; 0]$, show ***all the matrixes*** involved in computing the projection of point $\bar{p}_s$ onto the projector's image plane.

(typeset your answer to these questione into a .pdf, clearly labeled)

Part 1 – Projection mapping (continued)

One question that should come to your mind when you understand pinhole cameras (or cameras with a thin lens), is whether we can use a pinhole camera to ***magnify*** the objects it is imaging. This is what a ***microscope*** does.

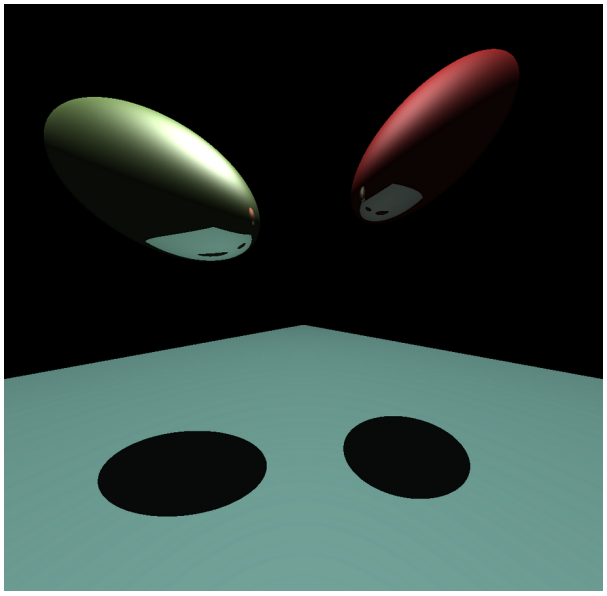
3) [3 marks] Given your answers to 1) and 2), provide ***pseudocode*** for a process that ***maps every point*** in the camera's (observer's) image plane to a corresponding point on the projector's image plane. Your algorithm should be ***general*** so it works with any projection surface geometry. Feel free to use ***functions*** as defined for the Whitted raytracer where appropriate to describe your algorithm

4) [up to 5 marks] ***crunchy*** – In a real setting, we can't use ray-tracing to figure out the mapping from points in the observer's image plane to pixels in the projector's image plane. ***Provide a step by step process*** (in pseudocode or using bullet points) that can be used to ***map pixels in the projector's image plane*** to ***pixels in the camera's image plane*** without ray tracing, and in a general setting where the projection surface could have any shape.

For full marks, your method must be complete, correct, and work for a projection surface with any shape.

(typeset all your written work for these questions into a .pdf, clearly labeled)

Build your own Ray Tracer



Output of my solution for this assignment

Basic Ray Tracer [45 marks in total]

Your task is to implement a basic ray-tracer and render a simple scene using ray casting and the Phong shading model as described in lecture. The starter code sets up a scene with two spheres and a plane as shown above, illuminated by a single point light source. Your job is to render the scene by implementing the code fragments missing for object intersections, Phong illumination, and recursive ray casting.

You will have to implement the following code fragments:

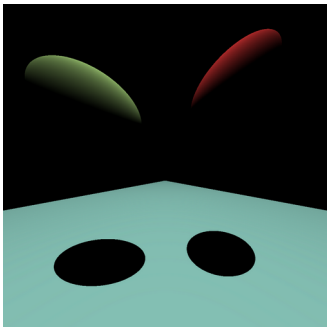
(a) ***[5 marks] Ray casting*** – computing a ray from the camera through each pixel and into the scene.
(b) ***[5 marks] Implement code for ray-sphere intersection***
(c) ***[5 marks] Implement code for ray-square intersection***
(d) ***[5 marks] Implement code for ray-cylinder intersection***
Note: All intersection code must work for affinely-deformed objects
(e) ***[5 marks] Compute the correct normals*** for affinely-deformed objects
(f) ***[10 marks] Phong illumination*** for a point light source (including ambient, diffuse, and specular components)
(g) ***[5 marks] Shadows***
(h) ***[5 marks] Reflection*** by recursive ray-casting

Note that your code will be auto-tested. ***DO NOT MODIFY THE API*** for the program, the original ***buildScene must work,*** and we must be able to ***change the scene by using the same commands to add objects and lights***. Any crunchy work ***must work without changing the original API.***

To demonstrate the working of your program, you must generate and submit along with your code the following images rendered at 512x512 pixels:



1. A scene signature where each pixel shows a unique color identifier for the first object hit (or background). This lets you test your ray casting/intersection code. You can generate this by making the ***ambient*** component to ***1.0***, and then setting the, ***diffuse***, ***specular, and global reflection*** components to zero.
   This image must be named: ***signature.ppm***



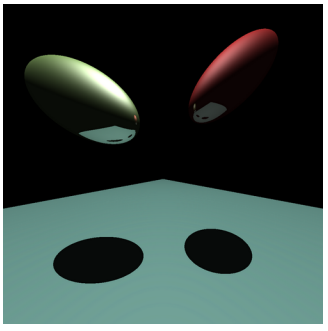
2. A rendered scene with only the diffuse and ambient components of the Phong model plus shadows. (set the ambient component for each object to ***0.1***)

   This image must be named: ***diffuse.ppm***



3. A rendered scene showing the specular component of the Phong model only (no ambient/diffuse). ***Note: Your highlights may look different*** from the image to the left (more spread out). That's normal, but ***they should be there, and have an oval shape***. If your highlights are circles, something is wrong with the computation of transformed normals.

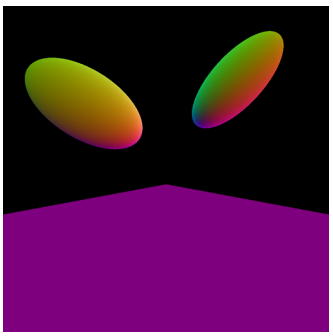
   This image must be named: ***specular.ppm***



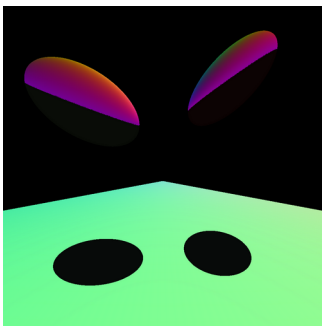
4. The final scene rendered with all components of the Phong model, including the global specular reflection Term.

   This image must be named: ***full.ppm***

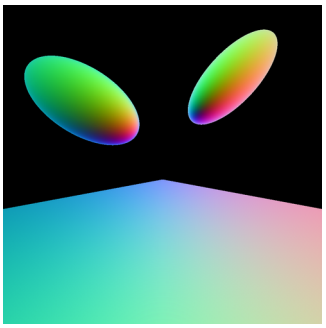
How to debug your code: Use Computer Graphics! You have a blank image, output to that image any quantities you need to look at. See the examples below, and make sure you understand what the images are telling you. The bottom line is: There is too much information going around inside the raytracer, printing values to the console will not be much help. An image can help you see what's going on if you use it well!



Example 1:
An image showing the components of the normal vector for each point on the objects in the scene.
R = (nx + 1)/2
G = (ny + 1)/2
B = (nz + 1)/2



Example 2:
An image showing the components of the ***r*** vector (perfect reflection direction) for each point in the scene



Example 3:
An image showing the components of the global reflection vector (the direction the ray hitting the objects is reflected toward).

You ***DO NOT*** have to submit images like the above, you are meant to use them to help you debug your code and make sure your computations are correct.

Remember, you can plot on the image pretty much any value being computed for a ray within your code! Use this extensively to ensure your code is working. It will save you time and help you find problems that are extremely hard to diagnose from printed values.

Once you notice something is not as it should, pick a pixel known to be in one of the spheres. Change the for loops so that the raytracer ***only*** processes that pixel. Print all values computed by your code (ray directions, intersection points, normals, and all vectors). Find and solve any problem(s). Start with a recursive depth of 1 and only increase it if everything else is working fine.

Compiling and running the starter code

Like all previous assignments. The raytracing code is designed to work on Linux. Therefore we expect you to do your work at the IC 406 lab. You can work remotely on Mathlab. This time the code will not produce a real-time visual display, so you do not have to worry about the graphical issues involved in working remotely over ssh. However, note that the code is fairly computationally intensive. You will be able to work faster if you are coding on a separate machine, as opposed to sharing the CPU with all other Mathlab users.

Use the included ***compile.sh*** script to compile the code.

Your final submission must compile and run without any form of glitch on Mathlab

Note: To encourage good coding practice, we will deduct up to

[10 marks] penalty - for badly designed code: that is, code lacking comments, confusing or unstructured code, or unnecessarily repetitive code.

Starter code in detail

utils.h, utils.c : Provide functions to manipulate points, vectors, matrices, and objects. This includes operations like dot products, matrix inversion, matrix-vector product, object creation and object transformations. There are parts in utils.c that you need to complete. Furthermore, you need to know the code in these functions since you will use these utilities extensively while building the raytracer.

RayTracer.h, RayTracer.c: These comprise the main code for the ray tracer. They follow the structure and flow of the ray tracer pseudo-code from the lecture notes.
You need to add code here to complete the ray tracer functionality.

svdDynamic.h, svdDynamic.c: This is a library used for matrix manipulation. You do not need to look at the code in any of these files.

Read carefully all the comments in the starter code – they describe in many places what you need to do. Parts of the code where work is needed are marked ***"TO DO"***.

– ***Unfortunately*** – We ***can not*** mark code that ***segfaults*** every time we run it so ***mind your use of memory***, and make sure that whatever work you turn it, it compiles, runs, and produces an image we can see

What to submit:

- ***ALL*** your code. That means all .h and .c files as well as the compilation script.

- The images requested above (signature.ppm, diffuse.ppm, specular.ppm, and full.ppm).

- Your written work for Part 1 in a PDF named ***A2_written_work.pdf***

- The scanned signatures page and a completed *autotester_id.txt*

Submitting your work

You will submit a single, compressed file named:

RayTracer_studentNo1.zip (e.g. RayTracer_11223344.zip)

Submit the resulting compressed file on Quercus after ***making sure it decompresses correctly and contains all the required files***.

General advice

- ***Start early:*** This is a fairly complex programming project, and requires both good C programming skills and understanding of the course material.

- ***Ask questions:*** Don't wait if you have problems. Ask your instructor or TA any questions related to the course material that arise while doing your work. If you have questions about the starter code, talk to the course instructor as well.

- ***Think, then code:*** The solution is reasonably short, but figuring out what the correct thing to do is requires thought and understanding. Do not start cobbling code together until you have understood what you should be doing.

- ***Have fun:*** Ray tracers are cool. After this assignment, you will understand how they work inside, and you will have your very own ray tracer to show off!

Marking Scheme	
Written part	18 marks
Ray tracer (you built one! This is awesome!)	45 marks
Crunchy bonus	Up to you!

Get Crunchy

Since you now have a working ray tracer, you may want to create a cool scene to show what it can do.

Possible extensions for bonus marks (have other ideas? Just send us a note! Go for it!)

[***Up to 10 marks***] – Create a very cool scene. Use reflection and shadows cleverly, together with a sufficiently deep recursion, reflective surfaces can create interesting visual effects. ***HOWEVER – no A.I. can be used for this part, it must be your own design!***

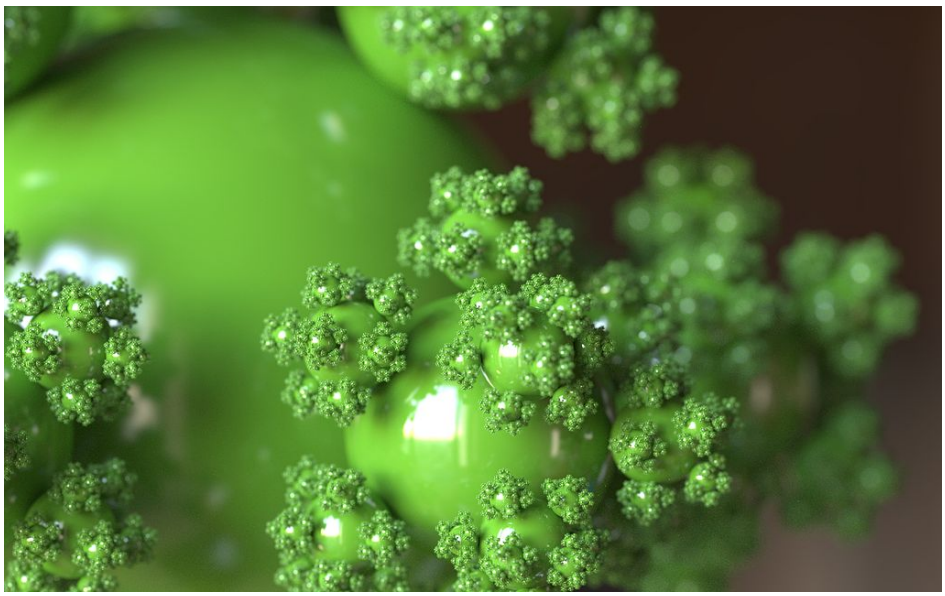
[5 marks] – Add support for morel canonical shapes, such as cones, boxes, and so on.

[up to 7 marks] – Introduce procedural geometry (full marks for a non-trivial implementation).

Build functions that take a few input parameters and create interesting geometry algorithmically. For inspiration, have a look at ***structure synth***. A rendering engine that uses very simple scripts to quickly create complex geometry.

Apply what you've learned! (remember we can build complex shapes using parametric curves, canonical objects, and transformations). Parametric curves can help you place objects along fairly complicated paths, and transforms will let you control their shape and orientation.

You could also look at L-systems – simple rule-based methods for producing structures that look like plants and trees.



Procedurally generated scene entirely made of spheres. A scene like this can easily be rendered (without the depth of Field effect) using your ray tracer
Image by Miles Bader, source: Wikipedia